



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

PHP

Trabajando con Formularios

Pepe

iNSTITUT



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE

Continguts

1	Trabajando con formularios	3
1.1	Las variables \$_GET, \$_POST y \$_REQUEST	3
2	Algunos envíos especiales	5
2.1	Recogida de campos múltiples	5
2.2	Envío de datos mediante enlaces	6
2.3	Envíos a la propia página	7

1 Trabajando con formularios

Como PHP se ejecuta dentro de HTML, sólo puede recibir datos del usuario de la aplicación a través del navegador web.

Y sólo hay una forma de introducir datos en una página web: a través de un formulario.

Veámoslo con un ejemplo. Supongamos que hemos definido en HTML este sencillo formulario:

```
<body>
  <form method="post" action="destino.php">
    Nombre<br/>
    <input type="text" name="nombre"><br/>

    Apellidos<br/>
    <input type="text" name="apellido"><br/>

    <input type="submit">
  </form>
</body>
```

Al pulsar el botón **Enviar**, se cargará el script **destino.php** en el servidor.

Ese script recibirá dos variables HTML llamadas **nombre y apellido**, con el valor que el usuario haya introducido en el formulario.

Para acceder a las variables HTML, se usa el array del sistema `$_POST` (En el siguiente epígrafe profundizaremos con los métodos), indexándolo con el nombre de la variable:

```
<?php
  echo "La variable nombre vale".$_POST['nombre']."<br>"
?>
```

1.1 Las variables `$_GET`, `$_POST` y `$_REQUEST`

Para recoger los datos que envían los clientes, tenemos predefinidas diferentes variables en PHP. Por ejemplo:

- si los datos se envían por método `GET` (cuando vienen de un enlace, o de un formulario con `method="get"`), podemos recogerlos en una variable llamada `$_GET`. Por el contrario,
- si se envían por método `POST` (para formularios con `method="post"`), podremos obtenerlos con la variable `$_POST`.
- Y tenemos una tercera variable, llamada `$_REQUEST`, que nos servirá para tomar los datos de cualquiera de estos dos métodos (y de otros más que no veremos aquí). **Así, lo más habitual**

será utilizar esta tercera variable, a no ser que queramos restringir la recepción de ciertos datos sólo a métodos GET o POST.

Cualquiera de estas tres variables es un array asociativo, es decir, un conjunto de datos enviados a los que se accede por el nombre de cada campo. Para acceder a un dato concreto de ese conjunto, en general, usaremos los corchetes y dentro, entre comillas, el mismo nombre que hayamos puesto en el atributo name del formulario que lo envió. Por ejemplo, si tenemos un formulario como este:

Y recogemos los datos en `destino.php`:

```
<?php
$nombre = $_GET["nombre"];
$apellido1 = $_GET["apellido"];
// Si cambiamos $_GET por $_POST o $_REQUEST, el resultado es el mismo.
// solo debéis fijaros en la URL.
echo "Hola $nombre $apellido";
?>
```

Siempre es interesante comprobar el estado de una variable antes de procesarla, para evitar algún disgusto. Los métodos se estudiaron en el apartado 2.3 *Comprobar el estado de las variables* del tema anterior.

Tomando como referencia el ejemplo anterior:

```
<?php
$nombre = $_REQUEST["nombre"];
$apellido = $_REQUEST["apellido"];

if(empty($nombre)){
    $nombre="Anónimo";
    $apellido="";
}
// Si cambiamos $_GET por $_POST o $_REQUEST, el resultado es el mismo.
// solo debéis fijaros en la URL.
echo "Hola $nombre $apellido";
?>
```

1.1.1 Si lo quisiéramos realizar todo en un único archivo (lo cual no es recomendable), podemos hacerlo así:

```
<form action="" method="get">
    <p><label for="nombre">Nombre: </label>
    <input type="text" name="nombre" id="nombre"></p>
    <p><label for="apellido1">Primer apellido:</label>
    <input type="text" name="apellido1" id="apellido1"></p>
    <input type="submit" value="enviar">
</form>
```

```
<p>
  <?php
    if(isset($_GET['nombre'])) {
      $nombre = $_GET["nombre"];
      $apellido1 = $_GET["apellido1"];

      echo "Hola $nombre $apellido1";
    }
  ?>
</p>
```

2 Algunos envíos especiales

En esta sección veremos cómo recoger datos que se envían de algún modo especial, como campos múltiples, ficheros, o datos a través de enlaces.

2.1 Recogida de campos múltiples

En el caso de campos con múltiples valores enviados (por ejemplo, una lista de selección múltiple), el campo del formulario tendrá este aspecto:

```
<form...>
...
  <select multiple name="personas[]" size="5">
    <option value="Juan Rodríguez">Juan Rodríguez</option>
    ...
  </select>
...
```

Al enviarse, recogeremos los elementos enviados en una variable que podremos recorrer con un *foreach* o una estructura similar:

```
$personas = $_REQUEST['personas'];

foreach ($personas as $persona) {
  ...
}
```

- Ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
<form action="" method="get">

<select name="lenguajes[]" multiple="true">
  <option value="c">C</option>
  <option value="java">Java</option>
  <option value="php">PHP</option>
  <option value="python">Python</option>
</select>
<br/>
<input type="checkbox" name="lenguajes[]" value="c" /> C<br />
<input type="checkbox" name="lenguajes[]" value="java" /> Java<br />
<input type="checkbox" name="lenguajes[]" value="php" /> Php<br />
<input type="checkbox" name="lenguajes[]" value="python" /> Python<br />
<input type="submit" value="enviar">
</form>
<?php

if(isset($_GET["lenguajes"])) {
    $lenguajes = $_GET["lenguajes"];

    foreach ($lenguajes as $lenguaje)
        echo "$lenguaje <br />";
}
?>
</body>
</html>
```

2.2 Envío de datos mediante enlaces

Hemos dicho que a través de los enlaces también podemos enviar datos al servidor. Normalmente, los enlaces no envían nada más que la página o recurso que queremos ver (por ejemplo, <http://www.google.es>). Pero también podemos añadir, al final de la URL, nombres de parámetros y sus respectivos valores, separados por el símbolo &, como si los enviáramos desde un formulario. Así, si quisiéramos “simular” el envío del formulario anterior para un usuario con login “usu1” y e-mail “usu1@gmail.com”, pondríamos un enlace así:

```
<a href="mipagina.php?login=usu1&email=usu1@gmail.com">
  Enviar datos
</a>
```

Estos datos podremos **recogerlos** a través de las variables `$_GET` o `$_REQUEST` (el método POST no se emplea en los enlaces):

```
$mailUsuario = $_GET["email"];
```

2.3 Envíos a la propia página

Es habitual también encontrarnos con formularios cuyo action apunta a la misma página del formulario.

Después, con código PHP (con instrucciones if..else), podemos distinguir si queremos cargar una página normal, o si hemos recibido datos del formulario y hay que procesarlos.

En cualquier caso, para poder hacer que un formulario se envíe a su propia página, podemos directamente poner el nombre de la misma página en el action (por ejemplo, `action="mipagina.php"`), pero si más adelante decidimos cambiar el nombre del archivo, corremos el riesgo de olvidarnos cambiar el formulario también. Para evitar este problema, contamos en PHP con una variable llamada `$_SERVER['PHP_SELF']` con lo que bastaría con poner esa variable en el action del formulario :

```
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
    ...
</form>
```

Si nos centramos en el array `$_SERVER` podemos consultar las siguientes propiedades:

- `PHP_SELF`: nombre del script ejecutado, relativo al document root (p.ej: /tienda/carrito.php)
- `SERVER_SOFTWARE`: (p.ej: Apache)
- `SERVER_NAME`: dominio, alias DNS (p.ej: www.elche.es)
- `REQUEST_METHOD`: GET
- `REQUEST_URI`: URI, sin el dominio
- `QUERY_STRING`: todo lo que va después de ? en la URL (p.ej: heroe=Batman&nombre=Bruce)

Más información en <https://www.php.net/manual/es/reserved.variables.server.php>